PYTHON LISTS

Logical Programming Questions

1. Given students' marks, create a list of marks greater than or equal to 35.

Sample Input	Expected Output
`marks = [28, 35, 67, 90, 31, 45]`	`Passed marks: [35, 67, 90, 45]`

2. Apply a 10% discount to product prices greater than ₹1,000. Leave other prices unchanged.

Sample Input	Expected Output
`prices = [500, 1200, 2500, 900, 1500]`	`Updated prices: [500, 1080.0, 2250.0, 900, 1350.0]`

3. Find the temperatures that are above the average temperature.

Sample Input	Expected Output
`temperatures = [28, 30, 32, 26, 34]`	`Average: 30.0` `Above average: [32, 34]`

4. Find the salaries greater than the median salary. For an even number of salaries, use the average of the two middle values after sorting.

Sample Input	Expected Output
`salaries = [30000, 45000, 25000, 60000, 40000]`	`Median: 40000` `Above median: [45000, 60000]`

5. Given matching lists of quantities and unit prices, calculate the total bill.

Sample Input	Expected Output
`quantities = [2, 3, 1]` `prices = [100, 50, 250]`	`Total bill: 600`

6. Remove duplicate student names while preserving their first-seen order.

Sample Input	Expected Output
`names = ['Asha', 'Ravi', 'Asha', 'Kiran', 'Ravi']`	`Unique names: ['Asha', 'Ravi', 'Kiran']`

7. Separate positive transactions (deposits) from negative transactions (withdrawals).

Sample Input	Expected Output
`transactions = [1000, -250, 500, -100, 750, -300]`	`Deposits: [1000, 500, 750]` `Withdrawals: [-250, -100, -300]`

8 Find the top three distinct scores without using sort() or sorted().

Sample Input	Expected Output
`scores = [78, 95, 88, 95, 72, 91, 88, 84]`	`Top three scores: [95, 91, 88]`

9. Find the longest contiguous run in which each number is greater than the previous number.

Sample Input	Expected Output
`numbers = [1, 2, 5, 3, 4, 6, 2, 8]`	`Longest increasing run: [3, 4, 6]`

10.Find the maximum sum of any non-empty contiguous sublist.

Sample Input	Expected Output
`numbers = [-2, 1, -3, 4, -1, 2, 1, -5, 4]`	`Maximum sublist sum: 6` `Sublist: [4, -1, 2, 1]`

11. Sort the list in ascending order without sort() or sorted().

Sample Input	Expected Output
`numbers = [5, 2, 9, 1, 5, 6]`	`Ascending: [1, 2, 5, 5, 6, 9]`

12. Sort descending without built-ins

Question: Sort the list in descending order without built-in sorting methods.

Sample Input	Expected Output
`numbers = [12, 4, 19, 7, 2]`	`Descending: [19, 12, 7, 4, 2]`

13 Find the second-largest distinct element without max() or sorting.

Sample Input	Expected Output
`numbers = [10, 5, 8, 10, 7, 8]`	`Second-largest distinct value: 8`

14. Find the second-smallest distinct element without min() or sorting.

Sample Input	Expected Output
`numbers = [10, 5, 8, 5, 7, 9]`	`Second-smallest distinct value: 7`

15. Find the third-largest element. Treat duplicate values as separate elements.

Sample Input	Expected Output
`numbers = [12, 5, 8, 12, 9, 3]`	`Third-largest element: 9`

16. Find all pairs of values whose sum equals the target. Each pair should use values at different positions.

Sample Input	Expected Output
`numbers = [2, 4, 3, 5, 7, 8, 1]` `target = 9`	`Pairs: [(2, 7), (4, 5), (8, 1)]`

17. Find unique value triplets whose sum is zero.

Sample Input	Expected Output
`numbers = [-1, 0, 1, 2, -1, -4]`	`Triplets: [(-1, -1, 2), (-1, 0, 1)]`

18. The list contains distinct numbers from 1 to n with one number missing. Find it.

Sample Input	Expected Output
`numbers = [1, 2, 4, 5, 6]`	`Missing number: 3`

`n = 6`	

19. Find values appearing in both lists without using sets. Return each common value once.

Sample Input	Expected Output
`a = [1, 2, 3, 4, 2]` `b = [3, 4, 4, 5, 1]`	`Common values: [1, 3, 4]`

20. Combine two lists and keep each value only once, preserving first-seen order.

Sample Input	Expected Output
`a = [1, 2, 3, 2]` `b = [3, 4, 5, 1]`	`Union: [1, 2, 3, 4, 5]`

Section 3: Nested Lists and Matrices

51. Find the sum of all numbers in a nested list.

Sample Input	Expected Output
`matrix = [[1, 2, 3], [4, 5], [6, 7]]`	`Sum: 28`

52. Find the largest number across all inner lists.

Sample Input	Expected Output
`matrix = [[12, 5], [7, 25], [9, 18]]`	`Largest element: 25`

53. Convert a one-level nested list into a single list without list comprehension.

Sample Input	Expected Output
`nested = [[1, 2], [3, 4, 5], [6]]`	`Flattened list: [1, 2, 3, 4, 5, 6]`

54. Flatten a one-level nested list using list comprehension.

Sample Input	Expected Output
`nested = [[10, 20], [30, 40], [50]]`	`Flattened list: [10, 20, 30, 40, 50]`

55: Print the main diagonal elements of a square matrix.

Sample Input	Expected Output
`matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`	`Main diagonal: [1, 5, 9]`

56 Calculate the sum of each row and each column.

Sample Input	Expected Output
`matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`	`Row sums: [6, 15, 24]` `Column sums: [12, 15, 18]`

57. Create the transpose of a matrix by converting rows into columns.

Sample Input	Expected Output
`matrix = [[1, 2, 3], [4, 5, 6]]`	`Transpose: [[1, 4], [2, 5], [3, 6]]`

58. Add two matrices of the same dimensions element by element.

Sample Input	Expected Output
A = [[1, 2], [3, 4]] B = [[5, 6], [7, 8]]	Sum matrix: [[6, 8], [10, 12]]

59.Find the row with the largest sum and report its one-based row number.

Sample Input	Expected Output
matrix = [[1, 2, 3], [10, 1, 1], [4, 5, 6]]	Row number: 3 Row: [4, 5, 6] Row sum: 15

60. Use nested list comprehension to extract all even numbers.

Sample Input	Expected Output
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 10]]	Even numbers: [2, 4, 6, 8, 10]

List Comprehension Challenges

61 Generate a list of squares of integers from 1 through 20.

Sample Input	Expected Output
Input: start = 1, end = 20	Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400]

62. Generate cubes of odd numbers from 1 through 30.

Sample Input	Expected Output
Input: start = 1, end = 30	Cubes: [1, 27, 125, 343, 729, 1331, 2197, 3375, 4913, 6859, 9261, 12167, 15625, 19683, 24389]

63. Extract numbers divisible by both 3 and 5.

Sample Input	Expected Output
numbers = [10, 15, 20, 30, 45, 50, 60, 72]	Result: [15, 30, 45, 60]

64. Extract words whose length is greater than five.

Sample Input	Expected Output
words = ['python', 'list', 'comprehension', 'code', 'student']	Result: ['python', 'comprehension', 'student']

65. Convert every string in a list to uppercase using list comprehension.

Sample Input	Expected Output
words = ['python', 'List', 'hello world']	Result: ['PYTHON', 'LIST', 'HELLO WORLD']

66. Extract all numbers greater than the list average.

Sample Input	Expected Output
`numbers = [10, 20, 30, 40, 50]`	`Average: 30.0` `Result: [40, 50]`

67. Create a list containing 'Even' or 'Odd' for each number.

Sample Input	Expected Output
`numbers = [1, 2, 3, 4, 5, 6]`	`Result: ['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even']`

68. Extract vowels from a string into a list, preserving their order and repeated occurrences.

Sample Input	Expected Output
`text = 'Programming in Python'`	`Vowels: ['o', 'a', 'i', 'o']`

69. Generate all ordered pairs by taking one value from each list.

Sample Input	Expected Output
`a = [1, 2]` `b = ['x', 'y']`	`Pairs: [(1, 'x'), (1, 'y'), (2, 'x'), (2, 'y')]`

70. Flatten a matrix and extract only positive values using nested list comprehension.

Sample Input	Expected Output
`matrix = [[-2, 3, 0], [4, -5, 6], [-1, 7, 8]]`	`Positive values: [3, 4, 6, 7, 8]`